APIs, Local Variables, Authentication and Prompt Engineering Concepts

PART A - PROGRAMMING AND WEB SERVICE CONCEPTS

1. What is an API?

An Application Programming Interface, commonly known as an API, is a set of rules, protocols, and tools that allows one software application to communicate with another. It defines the methods and data formats that applications can use to request and exchange information, acting as a contract between the provider of a service and the consumer of that service. Rather than exposing the internal workings of a system, an API exposes only the specific functions or data that the developer chooses to make available, hiding the underlying complexity behind a simple, predictable interface.

In the context of web development, APIs are most commonly accessed over HTTP. A client application sends a request to a specific endpoint (a URL), specifying an HTTP method such as GET, POST, PUT, or DELETE, along with any required parameters or data. The server processes this request and returns a response, typically formatted as JSON or XML, containing the requested data or the result of the requested operation.

Common Types of APIs

Type	Description	Example Use Case
REST API	Uses standard HTTP methods and is stateless; resources are identified by URLs	Fetching user profile data from a server
SOAP API	Uses XML-based messaging with strict standards and built-in error handling	Enterprise banking and payment systems
GraphQL API	Allows clients to request exactly the data they need in a single query	Mobile apps needing flexible, efficient data fetching
WebSocket API	Enables continuous two-way communication between client and server	Live chat applications and real-time notifications

APIs are essential because they enable integration between different systems, allow developers to reuse existing functionality instead of building it from scratch, and form the backbone of modern software architecture, including mobile applications, cloud services, and third-party integrations such as payment gateways, maps, and social media logins.

2. Overview of Local Variables (Purpose and Usage)

A local variable is a variable that is declared within a specific block of code, such as inside a function, method, or loop, and is only accessible within that block. Once the block of code finishes executing, the local variable is destroyed and its memory is released, meaning it cannot be accessed from outside the function or block in which it was defined. This behavior is governed by what is known as the variable's **scope**, which determines where in a program a variable can be referenced.

2.1 Purpose of Local Variables

- **Encapsulation:** Local variables keep data confined to the function that needs it, preventing other parts of the program from accidentally reading or modifying it.
- **Memory efficiency:** Because local variables exist only for the duration of the function call, memory is automatically freed once the function completes, reducing unnecessary memory usage.
- **Avoiding naming conflicts:** Since local variables are isolated to their own scope, the same variable name can be reused across different functions without causing conflicts.
- **Improved readability and debugging:** Restricting a variable's visibility to a small block of code makes it easier to track how and where the variable is used, simplifying debugging.

2.2 Usage of Local Variables

Local variables are typically used to store temporary values needed only during a specific calculation or operation, such as loop counters, intermediate results of an expression, or temporary copies of data passed into a function. For example, inside a function that calculates the average of a list of numbers, variables used to store the running total and the count of items are declared as local variables, since they have no meaning or relevance once the function has returned its final result.

2.3 Local Variables vs. Global Variables

Aspect	Local Variable	Global Variable
Scope	Limited to the function/block where declared	Accessible throughout the entire program
Lifetime	Exists only while the function is executing	Exists for the entire runtime of the program
Memory Usage	Released once the function ends	Occupies memory for the program's full duration
Risk of Conflicts	Low, since names are isolated to the block	Higher, since the same name affects the whole program

3. Types of Authentication, Error Handling, and OAuth

3.1 Types of Authentication

Authentication is the process of verifying the identity of a user or system before granting access to a resource. Different mechanisms are used depending on the level of security, scalability, and convenience required.

Type	How It Works	Typical Use Case
Basic Authentication	Username and password are sent with every request, usually encoded in the request header	Simple internal tools or testing environments
API Key Authentication	A unique key is issued to each client and included with every request to identify it	Third-party services like weather or maps APIs
Token-Based Authentication	A server issues a token after login, which the client then sends with subsequent requests	Modern web and mobile applications
JWT (JSON Web Token)	A self-contained, signed token that encodes user identity and claims, verifiable without a database lookup	Stateless APIs and microservices
Session-Based Authentication	The server creates and stores a session after login, identified by a session ID stored in a cookie	Traditional web applications
OAuth 2.0	A delegated authorization framework allowing a user to grant limited access to their resources without sharing credentials	'Sign in with Google/Facebook' style logins

3.2 Error Handling

Error handling refers to how a system detects, communicates, and responds to problems that occur during a request, such as invalid input, authentication failures, or server issues. In APIs, errors are typically communicated using standard HTTP status codes along with a structured response body that explains what went wrong.

Status Code	Meaning	Common Cause
400 Bad Request	The request is malformed or contains invalid data	Missing required fields or invalid data format
401 Unauthorized	Authentication has failed or is missing	Invalid, missing, or expired token
403 Forbidden	The client is authenticated but not allowed to access the resource	Insufficient permissions or scope
404 Not Found	The requested resource does not exist	Incorrect endpoint URL or deleted resource
500 Internal Server Error	An unexpected error occurred on the server	Bugs, crashes, or unhandled exceptions

3.3 OAuth (Open Authorization)

OAuth is an open standard for access delegation, commonly used to allow users to grant websites or applications access to their information on other services without giving away their passwords. Instead of sharing credentials directly, the user authorizes the application, and the authorization server issues an access token that the application uses to access the protected resources on the user's behalf, often with a

limited scope and expiry time.

The typical OAuth 2.0 flow involves four roles: the **Resource Owner** (the user), the **Client** (the application requesting access), the **Authorization Server** (which authenticates the user and issues tokens), and the **Resource Server** (which hosts the protected data and validates the access token before responding to requests). This separation allows access to be granted, limited, and revoked without ever exposing the user's actual login credentials to the requesting application.

PART B - PROMPT ENGINEERING CONCEPTS

1. Prompting Techniques

Prompting techniques are structured methods of designing the input given to an AI language model in order to guide it toward producing accurate, relevant, and well-structured responses. The way a prompt is framed has a direct impact on the quality, tone, and reliability of the output, making prompt design a critical skill when working with large language models.

Technique	Description
Zero-Shot Prompting	Asking the model to perform a task with no prior examples, relying entirely on its pre-trained knowledge.
Few-Shot Prompting	Providing a small number of examples within the prompt to demonstrate the desired format or pattern before asking the actual question.
Chain-of-Thought Prompting	Encouraging the model to reason through a problem step by step before arriving at a final answer, improving accuracy on complex tasks.
Role-Based Prompting	Assigning the model a specific persona or role (e.g., 'You are a senior data analyst') to shape the tone and expertise of the response.
Instruction-Based Prompting	Giving direct, explicit instructions about the task, format, and constraints the output should follow.
Iterative / Refinement Prompting	Building on previous responses through follow-up prompts to progressively refine and improve the output.

2. Common (Shorthand) Types of Prompts

Beyond formal techniques, prompts are also commonly classified by their everyday purpose or format, especially in productivity and content-generation contexts. These represent the typical 'shapes' a prompt can take depending on what the user wants to achieve:

- **Instructional Prompts:** Direct commands telling the model exactly what to do, such as 'Summarize this article in three bullet points.'
- **Completion Prompts:** Provide a partial sentence or text and ask the model to continue it naturally.
- **Question-Answer Prompts:** Pose a direct question expecting a factual or explanatory answer.
- **Conversational Prompts:** Open-ended, dialogue-style prompts used in chat interfaces that may build on earlier context.
- **Template / Format Prompts:** Specify a strict output structure, such as a table, JSON object, or specific report format.

- **Creative Prompts:** Request original content such as stories, poems, or marketing copy, often with stylistic constraints.

3. Prompt Refinement, Bias Parameters, and Support Parameters

3.1 Prompt Refinement (Resolvement)

Prompt refinement is the iterative process of improving a prompt based on the quality of the responses it produces. Rather than expecting a perfect result from the first attempt, the user evaluates the output, identifies where it falls short, such as being too vague, too long, missing context, or misunderstanding the intent, and then revises the prompt to address these gaps. This may involve adding more context, breaking a complex request into smaller steps, specifying the desired format more precisely, or providing examples of the expected output. Effective prompt refinement is essentially a feedback loop: prompt, evaluate response, adjust, and repeat until the output meets the required standard.

3.2 Bias Parameters

Bias, in the context of language models, refers to systematic tendencies in a model's output that may favor certain perspectives, demographics, or styles of answer over others, often as a result of patterns present in the data the model was trained on. Bias parameters refer to the considerations and controls used to identify, measure, and reduce such tendencies. This includes designing prompts that explicitly ask for balanced or neutral viewpoints, requesting multiple perspectives on a topic, avoiding leading or loaded language within the prompt itself, and being aware that the phrasing of a question can unintentionally steer the model toward a particular conclusion. Managing bias is important for ensuring that AI-generated content is fair, accurate, and does not reinforce stereotypes or one-sided narratives.

3.3 Support Parameters

Support parameters are the configurable settings that control how a language model generates its response, allowing the output to be tuned for creativity, length, consistency, and relevance. While the prompt defines *what* is being asked, these parameters influence *how* the model responds.

Parameter	Purpose
Temperature	Controls the randomness of the output; lower values produce more focused and deterministic responses, while higher values increase creativity and variability.
Top-p (Nucleus Sampling)	Limits the model's choices to the smallest set of words whose combined probability exceeds a threshold, balancing diversity and coherence.
Max Tokens	Sets the maximum length of the generated response, preventing overly long or truncated outputs.
Frequency Penalty	Reduces the likelihood of the model repeating the same words or phrases too often.
Presence Penalty	Encourages the model to introduce new topics or words rather than sticking closely to those already mentioned.
Stop Sequences	Defines specific text patterns that, when generated, signal the model to stop producing further output.

Note: Together, prompt refinement, awareness of bias, and careful tuning of support parameters allow users to move from a generic first response to an output that is accurate, balanced, and tailored to the exact needs of the task.